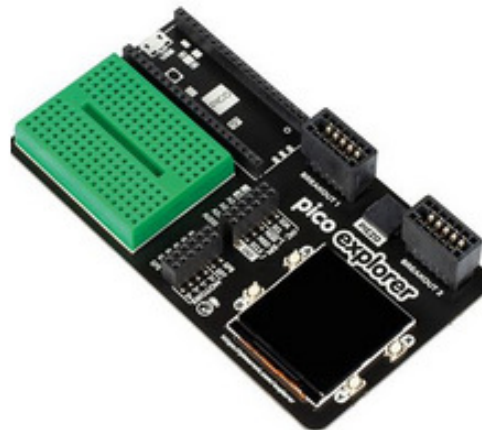


Pico-LCD-Module



Firmware Version 3.02 User Guide Updated June 17th, 2026

Add-on C-Language module to merge with your existing (C-Language)
Raspberry Pi Pico program / project to support
a Waveshare's 2-inch LCD display
or a Pimoroni's Pico-Explorer-Base LCD display.

IMPORTANT :

This User Guide is about Pico-LCD-Module Firmware Version 3.02 from Andre St. Louys.
To get the full potential of this module, read this User Guide and
carefully follow the indicated setup procedures.

Join our Pico-LCD-Module discussion group on:
<https://github.com/astlouys/Pico-LCD-Module/discussions>

Pico-LCD-Module User Guide

Index

Index	2
Introduction.....	3
Pico-LCD-Module setup.....	4
Setup part 1: Installing / copying the Pico-LCD-Module to your system.	4
Setup part 2: Integrating the Pico-LCD-Module to one of your own projects.	5
Pico-LCD-Example.....	8

Pico-LCD-Module User Guide

Introduction

Keep in mind that description, comments and source code apply to the hardware units that I had in hands by the time I wrote this guide. It must be understood that specifications may eventually have been modified in the product you bought.

That being said, both Waveshare's Pico 2-inch LCD display and Pimoroni's Pico Explorer Base LCD display use the same integrated circuit (ST7789) to control scanning and refresh of LCD display. Consequently, it is relatively easy to build a « library » to support both, except for a few differences...

First one is the LCD display definition: it is 320 X 240 pixels for the Waveshare's product, while it is 240 X 240 pixels for Pimoroni's. The other main difference is that Pimoroni's product does not provide a brightness control, the backlight LEDs being directly to power (and not through a GPIO, as for Waveshare's). Of course, Waveshare's unit is « only » a LCD display whereas Pimoroni's offers extra devices (piezo, small protoboard, breakers sockets, etc...)

The Pico-LCD-Module is a C-Language add-on module that will allow your existing Raspberry Pi Pico C-Language program or project to use either a Waveshare's 2-inch Pico LCD display or a Pimoroni's Pico Explorer Base 1.54 inch LCD display. Personally, I found that the size of these displays is a minimum required to be really useful. Other smaller displays exist, of course, and may be interesting if we only need them to check a few values once in a while, but can be too small to be used regularly to check many parameters or for monitoring purposes...

To help you figure out the details on how to implement and use the Pico-LCD-Module, an example is included in the repository: « Pico-LCD-Example ». This is a basic C-Language program, itself making use of the Pico-LCD-Module. An overview of this simple program is given later in this User Guide.

In a way similar to other « Pico-xxx-Module » Firmwares in my Git Hub repositories, you must follow the instructions given in this User Guide to get the full potential of this « library ».

- 1) The « Setup part 1 » covers the steps required to install the Pico-LCD-Example to your system. Make sure everything is installed as required and that you can build and run the program without problem. You can then go on with the following step to merge the add-on Pico-LCD-Module to your own program.
- 2) The « Setup part 2 » covers the steps to follow when you want to add the Pico-LCD-Module to one of your own existing Raspberry Pi Pico C-Language program.

You can join other users of the Pico-LCD-Module on the discussion group on:

<https://github.com/astlouys/Pico-LCD-Module/discussions>

I can also be reached with my personal email address at:

astlouys@gmail.com

Pico-LCD-Module setup

Setup part 1: Installing / copying the Pico-LCD-Module to your system.

NOTE: The instructions below assume that your development system is a Raspberry Pi running the Raspberry Pi O.S. (previously called « Raspbian ») and that you're using Visual Studio Code. It also assumes that you followed the recommended naming and structure conventions for the directory structure on your development system. If this is not the case, you will have to adapt the instructions accordingly.

- 1) Create the project directory on your system:

```
mkdir /home/pi/pico/Pico-LCD-Module
```

- 2) Clone or copy all files from the Git Hub repository to the « /home/pi/pico/Pico-LCD-Module » project directory created in step 1 above.

- 3) While in the « /home/pi/pico/Pico-LCD-Module » project directory create a symbolic link pointing to the pico_sdk_import.cmake file. This way, if / when you update the Pico SDK, your project will always use the latest version when it is built:

```
ln -s /home/pi/pico/pico-sdk/external/pico_sdk_import.cmake
```

- 4) If it does not already exist, create a « UTILITIES » directory in the pico directory and move the file « baseline.h » from the Pico-LCD-Module project directory to the « /home/pi/pico/UTILITIES » directory. The baseline.h file contains basic definitions that I use in most projects. You may want to take a look at it and add your own definitions and / or pieces of code that you use often. When you update the « baseline.h » file in the UTILITIES directory, it is automatically updated in all your projects through the symbolic link you will create in the next step. Make sure you have the most recent version of the « baseline.h » file.

```
mkdir /home/pi/pico/UTILITIES
```

```
mv /home/pi/pico/Pico-LCD-Module/baseline.h /home/pi/pico/UTILITIES
```

- 5) While in the Pico-LCD-Module directory, create a symbolic link to the « baseline.h » file:

```
ln -s /home/pi/pico/UTILITIES/baseline.h
```

- 6) Move the file « get_pico_identifier.c » from the Pico-LCD-Module directory to the UTILITIES directory. The Pico Unique Identifier will be used with many other projects to securely identify the different devices in the SLASH Smart Home ecosystem. Make sure you have the most recent version of the « get_pico_identifier.c » file.

```
mv get_pico_identifier.c /home/pi/pico/UTILITIES
```

Pico-LCD-Module User Guide

- 7) Create a symbolic link in the Pico-LCD-Module directory as you did before for the baseline.h file:

```
ln -s /home/pi/pico/UTILITIES/get_pico_identifier.c
```
- 8) Repeat the steps 6 and 7 above for the files « input_string.c » and « log_printf.c ».
- 9) This completes the first part of the setup. At this point, make sure you are able to build and use the Pico-LCD-Example application. Refer to the section of this User Guide on how to use it.

Setup part 2: Integrating the Pico-LCD-Module to one of your own projects.

NOTE: The instructions below assume that your development system is a Raspberry Pi running the Raspberry Pi O.S. (previously called « Raspbian ») and that you're using Visual Studio Code. It also assumes that you followed the recommended naming and structure conventions for the directory structure on your development system. If this is not the case, you will have to adapt the instructions accordingly.

Make sure you carefully followed the instructions in the « Setup part 1 » above, since they are a prerequisite for this section. Before going further, you must make sure that you're able to build and run successfully the Pico-LCD-Example Firmware. In the instructions below, « Pico-My-Project » should be replaced by the directory representing your own project.

If you already merge another « Pico-XXX-Module » from one of my Git Hub repositories, you should already know the recipe used...

- 1) From your project directory: « /home/pi/pico/Pico-My-Project », create a symbolic link pointing to the required files:

```
ln -s /home/pi/pico/pico-sdk/external/pico_sdk_import.cmake
ln -s /home/pi/pico/Pico-LCD-Module/Pico-LCD-Module.c
ln -s /home/pi/pico/Pico-LCD-Module/Pico-LCD-Module.h
ln -s /home/pi/pico/UTILITIES/baseline.h
ln -s /home/pi/pico/UTILITIES/get_pico_identifier.c
ln -s /home/pi/pico/UTILITIES/input_string.c
ln -s /home/pi/pico/UTILITIES/log_printf.c
```
- 2) If you plan to display text on LCD display from your project, create symbolic links in your project file to the font files. (NOTE: To save flash memory space in your program, you may want to exclude font definitions that you do not plan to use.)

```
ln -s /home/pi/pico/Pico-LCD-Module/font*
```

Pico-LCD-Module User Guide

- 3) If you plan to use some image files from the Pico-LCD-Module directory, create a symbolic link to those image files as you see fit (remember that image files use a relatively large memory space in flash memory). For example:

```
ln -s /home/pi/pico/Pico-LCD-Module/WaterFall1.img
```

- 4) In your own project source file (« /home/pi/pico/Pico-My-Project.c »), add the include file « Pico-LCD-Module.h » file:

```
#include "baseline.h"
#include "Pico-LCD-Module.h"
```

- 5) In your project file: « /home/pi/pico/Pico-My-Project.c », add the function prototype for the following functions (along with the other function prototypes of your own program):

```
get_pico_identifier();
input_string();
log_printf();
```

You may simply open each corresponding « .c » file and make a copy-paste of the first function line, while not forgetting to add a semi-colon at the end for the function prototype.

- 6) In your project file: « /home/pi/pico/Pico-My-Project.c », declare the structure defined in the Pico-LCD-Module.h file as a global variable:

```
struct struct_lcd StructLCD;
```

- 7) In your project file: « /home/pi/pico/Pico-My-Project.c », declare the LCD device that you're using. Only ONE device must be declared. NOTE: You will see LCD_114 in a few places in the source code. This definition is for the Waveshare's 1.14 inch display, but as mentioned before, it is not supported for now.

```
#define LCD_TYPE LCD_200 // for Waveshare's 2-inch
```

Or

```
#define LCD_TYPE LCD_EXPLORER // for Pimoroni's Explorer Base
```

- 8) Declare the fonts as « extern » (the same way it is done in the Pico-LCD-Example Firmware). In fact, you can refer to the Pico-LCD-Example for any question that may arise while you integrate the Pico-LCD-Module to your firmware to see how the integration has been done.

- 9) Find the section « LCD setup » at the begin of Pico-LCD-Example.c and copy-paste it in your program while adapting its content if required.

- 10) Depending on your program, you may also want to copy-paste other parts / functions of « Pico-LCD-Example.c » to use them.

NOTE: The 4 buttons (on the Waveshare's or Pimoroni's device) are properly initialized with the LCD_init() function of the Pico-LCD-Module.

Pico-LCD-Module User Guide

NOTES about Pimoroni's Pico Explorer Base:

The active buzzer and the standalone LED are available for your project but they need to be connected to one of the available GPIO. The Pico-LCD-Module does not handle them in any way,

Pico-LCD-Module User Guide

Pico-LCD-Example

It is assumed that you already installed the Pico on the Waveshare's 2-inches Pico LCD display or on the socket of the Pimoroni's Pico Explorer Base. It is also assumed that you rebuilt the Firmware after specifying in the source code the specific device used (Waveshare's or Pimoroni's).

Build the Pico-LCD-Example application and download it to your Pico. The application will automatically be launched when uploaded to the Pico.

NOTE: During the different tests, it is often required that the user presses <Enter> to continue. For some reason, flushing the input buffer on the Pico doesn't seem to work as for other devices. So, if you press <Enter> too fast (ahead of when asked to do so), you may easily override the input buffer and need to restart the Firmware. This is something I will investigate in the future and eventually fix in a future version. (Let me know if you already had this problem and fix it).

The « Pico-LCD-Example » application is very basic... Its main purpose is to show how to integrate the « Pico-LCD-Module » to your current program and also to test the different functions offered by this module. When started, the Firmware will display a menu as shown below:

```
[ 1035 0] [log_header] - -----
[ 1036 0] [log_header] -                               Pico-LCD-Example
[ 1037 0] [log_header] -                               Firmware version 3.02   Built on Jun 20 2026 14:20:12
[ 1038 0] [log_header] -                               Compatible with the SLASH Smart Home ecosystem family
[ 1039 0] [log_header] -                               Pico unique ID: <E661-4103-E79D-5723>
[ 1040 0] [log_header] -                               Device identifier: <E661-4103-E79D-5723>
[ 1062 0] [log_header] -                               Configured for Waveshare's 2-inch device.
[ 1063 0] [log_header] -                               Current system timer value: 513265
[ 1065 0] [log_header] - -----
[  424 0] [main] -                               Main program entry point (Delay: 0 msec waiting for USB CDC connection).
[  425 0] [main] -                               Initializing LCD display in native portrait mode (Rotation: 0-degree / Mirroring: none).
[  426 0] [main] - -----

[ 1159 0] [term_menu] - -----
[ 1160 0] [term_menu] -                               Terminal menu
[ 1161 0] [term_menu] - -----
[ 1162 0] [term_menu] -                               1) - Show current LCD display settings.
[ 1163 0] [term_menu] -                               2) - Perform a hardware reset on LCD display and reinitialize it.
[ 1164 0] [term_menu] -                               3) - Perform a software reset on LCD display and reinitialize it.
[ 1165 0] [term_menu] -                               4) - Change rotation setting.
[ 1166 0] [term_menu] -                               5) - Mirror the framebuffer.
[ 1167 0] [term_menu] -                               6) - Reset LCD display in native mode.
[ 1168 0] [term_menu] -                               7) - Image analysis.
[ 1169 0] [term_menu] -                               8) - Testing function LCD_clear().
[ 1170 0] [term_menu] -                               9) - Testing function LCD_cls().
[ 1171 0] [term_menu] -                               10) - Testing function LCD_display_point().
[ 1172 0] [term_menu] -                               11) - Testing function LCD_draw_box().
[ 1173 0] [term_menu] -                               12) - Testing function LCD_draw_circle().
[ 1174 0] [term_menu] -                               13) - Testing function LCD_draw_image().
[ 1175 0] [term_menu] -                               14) - Testing function LCD_draw_line().
[ 1176 0] [term_menu] -                               15) - Testing function LCD_draw_point().
[ 1177 0] [term_menu] -                               16) - Testing function LCD_mirror_framebuffer().
[ 1178 0] [term_menu] -                               17) - Testing function LCD_part_clear().
[ 1179 0] [term_menu] -                               18) - Testing function LCD_part_cls().
[ 1180 0] [term_menu] -                               19) - Testing function LCD_part_refresh().
[ 1181 0] [term_menu] -                               20) - Testing function LCD_printf().
[ 1182 0] [term_menu] -                               21) - Testing function LCD_scroll().
[ 1183 0] [term_menu] -                               22) - Testing function LCD_set_brightness().
[ 1184 0] [term_menu] -                               23) - Testing function LCD_set_pixel().
[ 1185 0] [term_menu] -                               24) - Testing function LCD_set_rotation().
[ 1186 0] [term_menu] -                               25) - Testing function LCD_splash_image().
[ 1187 0] [term_menu] -                               26) - Proceed with all tests in sequence.
[ 1188 0] [term_menu] -                               27) - Jump to test zone.
[ 1189 0] [term_menu] -                               28) - Read LCD display parameters.
[ 1190 0] [term_menu] - -----
[ 1191 0] [term_menu] -                               77) - Clear terminal screen.
[ 1192 0] [term_menu] -                               88) - Restart the Firmware.
[ 1193 0] [term_menu] -                               99) - Switch Pico in upload mode.
[ 1194 0] [term_menu] - -----
[ 1195 0] [term_menu] -                               Enter your choice: 
```


Pico-LCD-Module User Guide

Most menu choices are self-explanatory. However, I suggest that you first go with menu number 24: « Testing function LCD_set_rotation() » to make sure you are familiar with « LCD display rotation » options.

When done, I recommend going on with option number 26: « Proceed with all tests in sequence ». This option will first set rotation to 0-degree and proceed with all tests available in the menu. Rotation is then changed to 90-degree and tests are executed again with this new rotation setting. 180-degree rotation is done next, and finally, rotation at 270-degree.